PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT – 3

Case Study 3: Smart Parking Fee Management

1. Problem Statement

Managing parking fees manually can lead to calculation errors, incorrect charges, and difficulties in tracking parking duration. This project aims to develop a Python program that calculates parking fees based on vehicle category, entry time, and exit time. The program considers overnight parking, a maximum daily fee limit, and penalty conditions. It also validates the entered time information to ensure accurate fee calculation and displays the final parking charges for the vehicle.

2. Objectives

1. To calculate parking fees based on the vehicle category and parking duration.
2. To accept vehicle entry and exit times in 24-hour format.
3. To calculate parking duration, including overnight parking.
4. To apply the maximum parking fee limit and overnight surcharge.
5. To validate time inputs and handle invalid entries.
6. To generate a clear parking fee report showing the total amount payable.

3. Assumptions

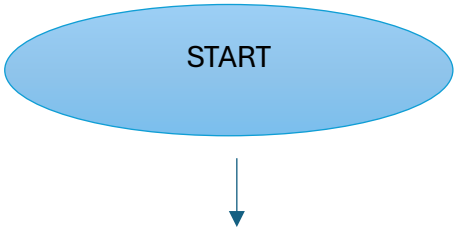
1. The parking fee for a car is ₹30 per hour.
2. The parking fee for a bike is ₹10 per hour.
3. Any started hour is charged as one full hour.
4. If the exit time is earlier than the entry time, the vehicle is assumed to have exited the next day.
5. An overnight surcharge of ₹20 is added when parking continues into the next day.

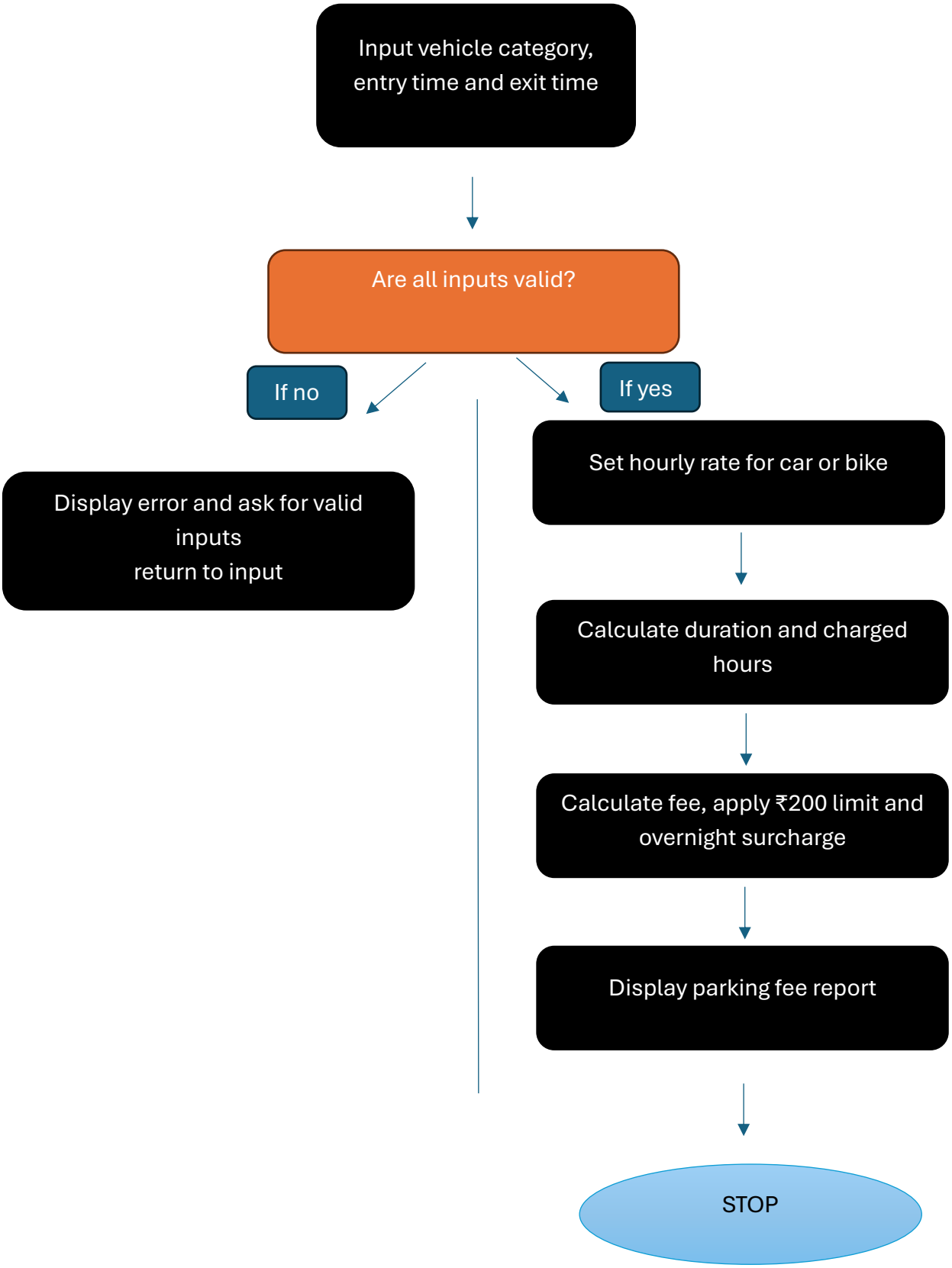
6. The maximum basic parking fee is limited to ₹200 per parking session. The overnight surcharge is added separately.
7. Entry and exit times must be entered in valid 24-hour HH:MM format.
8. Invalid vehicle categories or invalid time values are rejected, and the user must enter valid information again.
9. For this project, the parking duration is assumed to be less than 24 hours.

4. Algorithm

1. Start the program.
2. Input the vehicle category (car or bike).
3. Input the entry time and exit time in HH format.
4. Validate the vehicle category and time inputs.
5. If any input is invalid, display an error message and request valid input.
6. Assign the hourly parking rate based on the vehicle category.
7. Calculate the parking duration using the entry and exit times.
8. If the exit time is earlier than the entry time, consider the exit to be on the next day.
9. Round the parking duration up to the next whole hour if necessary.
10. Calculate the basic parking fee by multiplying the charged hours by the hourly rate.
11. If the basic fee exceeds ₹200, limit it to ₹200.
12. If the vehicle stays overnight, add the ₹20 overnight surcharge.
13. Calculate the total parking fee.
14. Display the vehicle category, parking duration, charged hours, basic fee, surcharge, and total fee.
15. Stop the program.

5: Flowchart




Input vehicle category, entry time and exit time
Are all inputs valid?
If no
If yes
Display error and ask for valid inputs
return to input
Set hourly rate for car or bike
Calculate duration and charged hours
Calculate fee, apply ₹200 limit and overnight surcharge
Display parking fee report
STOP

6. Pseudocode

```
START
INPUT vehicle _category
INPUT entry_ time
INPUT exit_ time
IF vehicle_ category is not "car" AND not "bike" THEN
DISPLAY "Invalid vehicle category"
ELSE
VALIDATE entry _time and exit _time

IF time inputs are invalid THEN

  DISPLAY "Invalid time format"

ELSE

  IF vehicle_ category = "car" THEN

    hourly_ rate = 30

  ELSE

    hourly_ rate = 10

  END IF

  CALCULATE parking duration
```

IF exit _time is earlier than entry _time THEN

CONSIDER exit as next day

overnight_ surcharge = 20

ELSE

Overnight _surcharge = 0

END IF

CALCULATE charged hours by rounding up

basic_ fee = charged_ hours * hourly _rate

IF basic_ fee > 200 THEN

basic_ fee = 200

END IF

total_ fee = basic _fee + overnight _surcharge

DISPLAY parking duration

DISPLAY basic_ fee

DISPLAY overnight _surcharge

DISPLAY total _fee

END IF

END IF

STOP

7.source code

PYTHON-ACTIVITY-BASED-LEARNING > Smart Parking Fee Management.py > ...

```python

vehicle = input("Enter vehicle type (car/bike): ").lower()
entry = int(input("Enter entry hour (0-23): "))
exit_time = int(input("Enter exit hour (0-23): "))

if vehicle == "car":
    rate = 30
elif vehicle == "bike":
    rate = 10
else:
    rate = 0
    print("Invalid vehicle type")

if rate > 0:
    if exit_time < entry:
        duration = (24 - entry) + exit_time
        surcharge = 20
    else:
        duration = exit_time - entry
        surcharge = 0

    hours = duration

    if duration > hours:
        hours = hours + 1

    basic_fee = hours * rate

    if basic_fee > 200:
        basic_fee = 200

    total_fee = basic_fee + surcharge

    print("Parking duration:", duration, "hours")
    print("Charged hours:", hours)
    print("Basic fee: Rs.", basic_fee)
    print("Overnight surcharge: Rs.", surcharge)
    print("Total fee: Rs.", total_fee)

```

8. Test Cases

Test Case	Input	Expected Output
TC1	Vehicle: car, Entry: 10:00, Exit: 12:00	Duration: 2 hours, Basic Fee: ₹60, Surcharge: ₹0, Total: ₹60
TC2	Vehicle: bike, Entry: 10:00, Exit: 10:30	Duration: 0.5 hours, Charged Hours: 1, Basic Fee: ₹10, Total: ₹10
TC3	Vehicle: car, Entry: 22:00, Exit: 02:00	Duration: 4 hours, Basic Fee: ₹120, Surcharge: ₹20, Total: ₹140
TC4	Vehicle: bus, Entry: 10:00, Exit: 11:00	Output: Invalid vehicle type.

9. Output Screenshots



10. Conclusion

The Smart Parking Fee Management program calculates parking charges based on vehicle type and parking duration. It applies hourly rates, rounds up partial hours, adds an overnight surcharge, and limits the basic parking fee to ₹200. The program also handles invalid vehicle types and time formats. This project demonstrates the use of Python variables, conditional statements, exception handling, and arithmetic operations in solving a real-world problem.